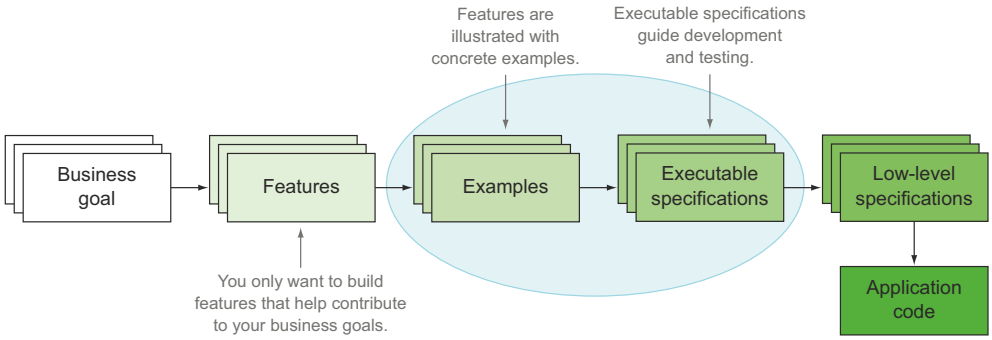


Figure 5.1 In this chapter we'll take examples we used to discuss and illustrate features in previous chapters and turn them into executable specifications.

and write executable specifications in a way that makes it easy to automate them. BDD has a number of well-defined practices to achieve this shared understanding:

- BDD practitioners express concrete examples as *executable scenarios*, using a semi-structured "given ... when ... then" format that's easy for both stakeholders and team members to read.
- This format can be automated using BDD tools such as JBehave, Cucumber, and SpecFlow.
- *Tables* can be used to combine several similar examples more concisely in a single scenario, or to express test data or expected results in a more succinct way.
- Experienced BDD practitioners take care to write their scenario steps well, providing enough detail for the scenario to be meaningful, but not so much that the essential business goals of the scenario are hard to find.
- Scenarios are organized in feature files and can be annotated with *tags* to indicate cross-functional concerns and to coordinate test execution.

It's important for everyone to be comfortable with the notation and structures used for these scenarios; that way, team members can focus on discussing the requirements and not be distracted by the form that you use to express them. As in any language, there are common patterns and structures that recur (idioms, so to speak), and those can help you express your ideas more fluently.

In the next chapter, you'll see how to automate these examples using popular BDD tools: JBehave (Java), Cucumber (Java and Ruby), SpecFlow (.NET), and Behave (Python). But if you want to experiment with the examples we discuss in this chapter, you can download the JBehave and Cucumber versions from GitHub (https://github.com/bdd-in-action/chapter-5) or the Manning website.

5.1 Turning concrete examples into executable scenarios

Imagine you're working on the Flying High Frequent Flyer application we discussed in chapters 3 and 4. Your job is to implement a feature that will allow Frequent Flyer